

MACHINE LEARNING SESSION

K-Means & Naive Bayes in Plain English

Two case studies, real datasets, and the questions students always ask about supervised vs. unsupervised learning

Today's roadmap

01

Supervised vs. Unsupervised

The question everyone asks first — answered with everyday scenarios

02

K-Means Clustering

Grouping mall customers by income and spending — no labels needed

03

Naive Bayes

Teaching a model to spot spam texts using probability

04

FAQs & Wrap-up

The questions students ask most, answered side by side

FREQUENTLY ASKED QUESTION

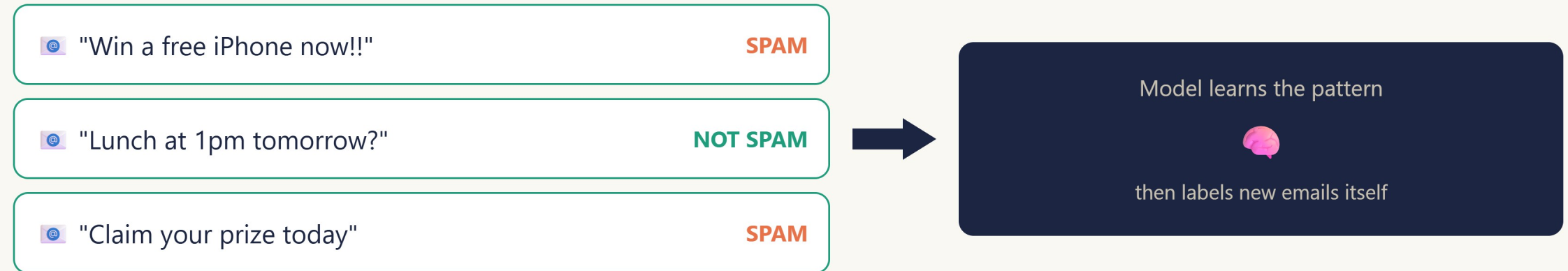
"What's the difference between supervised and unsupervised learning?"

Short answer: it comes down to whether the "right answers" already exist in your data. Let's look at it as two everyday scenes.

Scenario 1 – Supervised Learning

Labels already exist

Your inbox already has 2,000 old emails marked "Spam" or "Not Spam". You want a model that learns from these labels, then predicts the label for brand-new emails.

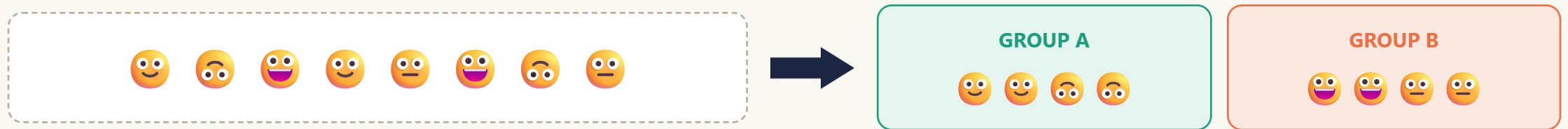


Supervised = learning FROM answers you already have

Scenario 2 — Unsupervised Learning

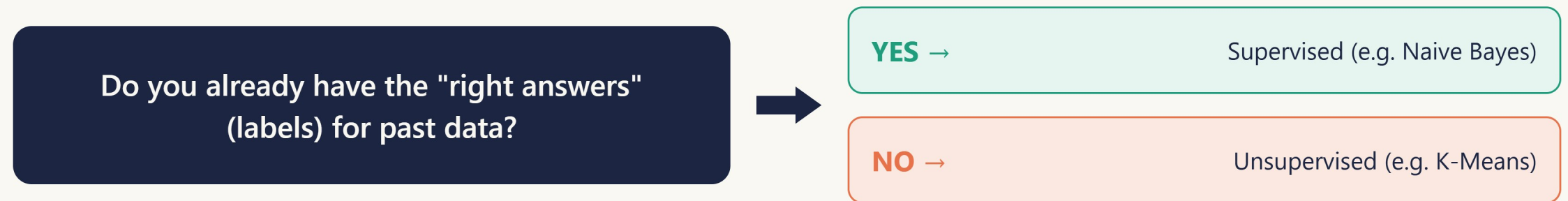
No labels at all

You have 500 customer photos in one folder — nobody tagged who is who or which "type" they belong to. You just want visually similar ones grouped together.



Unsupervised = finding structure WITHOUT being told the answer

FAQ — "How do I know which one to use?"



Rule of thumb: if your spreadsheet already has a "label" or "outcome" column, you're doing supervised learning.

PART 1

K-Means Clustering

Unsupervised · grouping by similarity · no labels required

A realistic example first

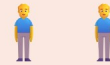
A clothing brand has height & weight for 10,000 customers but no size labels. K-Means groups similar body types together so the brand can design "S / M / L" fits — the groups emerge from the data itself.



Cluster 1 — "S"



Cluster 2 — "M"



Cluster 3 — "L"

Nobody told the algorithm what "S / M / L" means — it only grouped similar points, and we assigned the meaning afterward.

The algorithm, in 4 steps

1

Initialize

Pick k random points as starting centroids

2

Assign

Give every point to its nearest centroid

3

Update

Move each centroid to the mean of its group

4

Repeat

Redo steps 2–3 until centroids stop moving

Choosing k : run this for $k = 1 \dots 10$ and look for the "elbow" in the error curve — we'll do this live.

Case Study — Mall Customer Segments



What we saw ($k = 5$)

- High income + high spending → "VIP" targets
- High income + low spending → untapped, needs marketing
- Low income + high spending → credit-risk watch
- Low income + low spending → budget shoppers
- Middle of both → the "average" majority

The elbow method confirmed $k = 5$ gave the clearest groups

PART 2

Naive Bayes

Supervised · probability-based classification · learns from labeled examples

A realistic example first

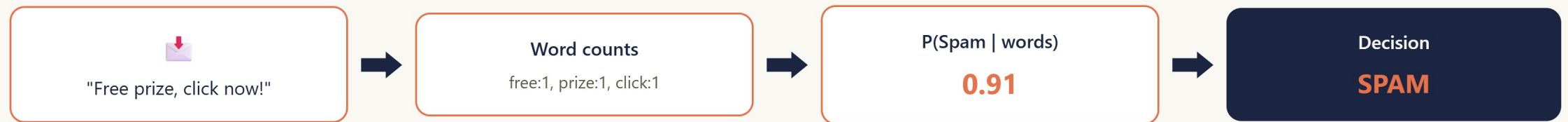
You see someone carrying an umbrella 🌂 on a sunny-looking morning. Weird? You instantly combine two things: how often people carry umbrellas at all, and how often umbrella-carriers turn out to be right about rain. That combining is exactly Bayes' Theorem.

$$P(\text{Rain} \mid \text{Umbrella}) = P(\text{Umbrella} \mid \text{Rain}) \times P(\text{Rain}) \div P(\text{Umbrella})$$

Posterior = Likelihood × Prior ÷ Evidence

Naive Bayes applies exactly this formula to classify things like emails or text messages — just with word counts instead of umbrellas.

How the model decides



Compare $P(\text{Spam} \mid \text{words})$ vs. $P(\text{Not Spam} \mid \text{words})$ — pick whichever is higher. That's the entire prediction step.

Case Study — SMS Spam Detector

Confusion matrix (test set)

	Pred. Ham	Pred. Spam
Actual Ham	955	3
Actual Spam	18	139

✅ ~98% overall accuracy

✅ Only 3 real messages wrongly flagged as spam — low "annoyance" cost

⚠️ 18 spam messages slipped through as ham — spam recall matters more than raw accuracy

🎯 With 87% ham / 13% spam, a lazy model could hit 87% by guessing "ham" every time

Questions students always ask

Q: Can K-Means predict labels like Naive Bayes?

No — it only groups; it never learns a target class

Q: Why is it called "naive"?

It assumes features don't affect each other — rarely true, works anyway

Q: Do I need to scale my data?

Always for K-Means (distance-based); usually not for Naive Bayes

Q: How do I pick k in K-Means?

Elbow method + silhouette score — never guess blindly

Side by side

	K-Means	Naive Bayes
Learning type	Unsupervised	Supervised
Needs labels?	No	Yes
Output	Cluster number	Class + probability
Real example	Customer segments	Spam filter

Questions?

Try it yourself: swap `CountVectorizer` for `TfidfVectorizer`, or re-run K-Means with a third feature — what changes?